NLA Note

1 Background

Floating Point ($\mathbb{F}$): A number = mantissa $\times$ base$^{\text{exponent}}$ (e.g. $2048 = 1 \times 2^{11}$ mantissa=1, base=2, exponent=11)

Assumptions: There is $\varepsilon_m > 0$ **(machine epsilon)** **A1**: $\forall \alpha \in \mathbb{R}, \exists \varepsilon \in (-\varepsilon_m, \varepsilon_m)$ s.t. $fl : \mathbb{R} \to \mathbb{F}$ by $fl(\alpha) = \alpha(1+\varepsilon)$

A2: $\forall \alpha, \beta \in \mathbb{F}, \forall * \in \{+, -, \times, \div\}, \exists \varepsilon \in (-\varepsilon_m, \varepsilon_m)$ s.t. $\alpha \circledast \beta = (\alpha * \beta)(1+\varepsilon)$

A1: The rounding error in α is relative to the size of α ($|\alpha - fl(\alpha)| < |\alpha|\varepsilon_m$) A2: The error in $\alpha \circledast \beta$ is relative to the size of $\alpha * \beta$ ($|\alpha \circledast \beta - \alpha * \beta| \le |\alpha * \beta|\varepsilon_m$)

IEEE (Double Precision): **64-bit**: denote $\mathbb{F}$. 1-bit: sign 52-bit: mantissa 11-bit: exponent (base 2). $\varepsilon_m \approx 2.22 \times 10^{-16}$

Cost of Algorithm (C): $C :=$ number of floating point operations (FLOPs) If $C(n) \le \alpha f(n), \Rightarrow C(n) = \mathcal{O}(f(n))$. n is problem size

· We consider that: 交换/赋值 no cost (i.e. Permutation **P** is free), 比较两数 1 FLOPs, $\sqrt{\ }$ 1 FLOPs.

Norms: $\|\mathbf{x}\|_p := (\sum_{i=1}^n |x_i|^p)^{1/p}, p \ge 1$ $\|\mathbf{x}\|_\infty := \max_{1\le i\le n} |x_i|$ $\|\mathbf{x}\|_2 := (\sum_{i=1}^n |x_i|^2)^{1/2}$ $\|\mathbf{x}\|_1 := \sum_{i=1}^n |x_i|$

1. $\|\mathbf{x}\|_p \ge 0$ $\|\mathbf{x}\|_p = 0 \Leftrightarrow \mathbf{x} = \mathbf{0}$
2. $\|\alpha\mathbf{x}\|_p = |\alpha|\|\mathbf{x}\|_p$ for $\alpha \in \mathbb{R}$
3. $\|\mathbf{x}+\mathbf{y}\|_p \le \|\mathbf{x}\|_p + \|\mathbf{y}\|_p$ (Triangle inequality)
4. **Induced Norm**: $\|\mathbf{A}\|_p := \max_{\mathbf{x}\neq 0} \frac{\|\mathbf{Ax}\|_p}{\|\mathbf{x}\|_p}$

1. $\|\mathbf{A}_{m\times n}\|_\infty := \max_{1\le j\le m} \sum_{i=1}^n |a_{ij}|$ (maximum row sum)
2. $\|\mathbf{A}_{m\times n}\|_1 := \max_{1\le i\le n} \sum_{j=1}^m |a_{ij}|$ (maximum column sum)
3. $\|\mathbf{A}_{m\times n}\|_2 := \sqrt{\rho(\mathbf{A}^\top\mathbf{A})}$ $\rho(\mathbf{A}^\top\mathbf{A}) = \max\{|\lambda| : \lambda \text{ is eigenvalue of } \mathbf{A}^\top\mathbf{A}\}$

2 Methods for Systems of Linear Equations (SLE)

Ideal: We want to solve SLE: $\mathbf{Ax} = \mathbf{b}$, where $\mathbf{A}_{n\times n}$ is invertible, $\mathbf{x}, \mathbf{b} \in \mathbb{R}^n$

2.1 Backward Stable | Condition Number | Error

Backward Stable: An algorithm to solve SLE is Backward Stable if: $\exists \alpha(n) > 0$ s.t. $\forall \mathbf{A}, \mathbf{b}$, the computed solution $\hat{\mathbf{x}}$ satisfies:

$$(\mathbf{A}+\Delta\mathbf{A})\hat{\mathbf{x}} = \mathbf{b} + \Delta\mathbf{b} \quad \text{and} \quad \frac{\|\Delta\mathbf{A}\|_p}{\|\mathbf{A}\|_p} + \frac{\|\Delta\mathbf{b}\|_p}{\|\mathbf{b}\|_p} \le \alpha\varepsilon_m$$

ε_m : machine epsilon. ps: α 可以和问题规模 n 相关, 但不能和 **A** 或 **b** 相关.

Condition Number ($\kappa_p(\mathbf{A})$): $\kappa_p(\mathbf{A}) := \begin{cases} \|\mathbf{A}\|_p\|\mathbf{A}^{-1}\|_p \ge 1 & \text{if } \mathbf{A} \text{ is invertible} \\ \infty & \text{otherwise} \end{cases}$ $\begin{cases} \text{If } \kappa_p(\mathbf{A}) \text{ is large (e.g. } \kappa_p(\mathbf{A}) \ge 10^{12} \approx \varepsilon_m^{-1}), & \mathbf{A} \text{ is ill-conditioned.} \\ \text{If } \kappa_p(\mathbf{A}) \le 10^4, & \mathbf{A} \text{ is well-conditioned.} \end{cases}$ ps: $\kappa_p(\mathbf{A})$ 衡量矩阵奇异性

1. For $\mathbf{A}_{n\times n}$ with real eigenvalues $|\lambda_1| \ge |\lambda_2| \ge \cdots \ge |\lambda_n|$ and corresponding eigenvectors $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n$: ps: 对所有实对称矩阵都满足下面条件

   If $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n$ are orthonormal set. Then $\kappa_2(\mathbf{A}) = \|\mathbf{A}\|_2\|\mathbf{A}^{-1}\|_2 = \frac{|\lambda_1|}{|\lambda_n|} \ge 1$

Relative Error: $\frac{\|\hat{\mathbf{x}}-\mathbf{x}\|_p}{\|\mathbf{x}\|_p}$ **Actual Error:** $\|\hat{\mathbf{x}}-\mathbf{x}\|_p$ ps: Relative error gives more information than actual error.

1. Let $\mathbf{Ax} = \mathbf{b}$ and $(\mathbf{A}+\Delta\mathbf{A})\hat{\mathbf{x}} = \mathbf{b}$. If $\kappa_p(\mathbf{A})\frac{\|\Delta\mathbf{A}\|_p}{\|\mathbf{A}\|_p} < 1$ then $\frac{\|\mathbf{x}-\hat{\mathbf{x}}\|_p}{\|\mathbf{x}\|_p} \le \frac{\kappa_p(\mathbf{A})}{1-\kappa_p(\mathbf{A})\frac{\|\Delta\mathbf{A}\|_p}{\|\mathbf{A}\|_p}} \cdot \frac{\|\Delta\mathbf{A}\|_p}{\|\mathbf{A}\|_p}$. ps: $\frac{\|\Delta\mathbf{A}\|_p}{\|\mathbf{A}\|_p}$ $\frac{\|\Delta\mathbf{b}\|_p}{\|\mathbf{b}\|_p}$ 小代表 good stability.
2. Let $\mathbf{Ax} = \mathbf{b}$ and $(\mathbf{A}+\Delta\mathbf{A})\hat{\mathbf{x}} = \mathbf{b}+\Delta\mathbf{b}$. If $\kappa_p(\mathbf{A})\frac{\|\Delta\mathbf{A}\|_p}{\|\mathbf{A}\|_p} < 1$ then $\frac{\|\mathbf{x}-\hat{\mathbf{x}}\|_p}{\|\mathbf{x}\|_p} \le \frac{\kappa_p(\mathbf{A})}{1-\kappa_p(\mathbf{A})\frac{\|\Delta\mathbf{A}\|_p}{\|\mathbf{A}\|_p}} \cdot (\frac{\|\Delta\mathbf{A}\|_p}{\|\mathbf{A}\|_p} + \frac{\|\Delta\mathbf{b}\|_p}{\|\mathbf{b}\|_p})$.

2.2 LU Factorization | Gaussian Elimination (GE)

Def of LU: If $\mathbf{A}_{n\times n}$ is invertible, then $^1\exists$ **L**: 单位下三角, $^2\exists$ **U**: 可逆上三角. s.t. $\mathbf{A} = \mathbf{LU}$ **(unique)**

LU Factorization: By Gaussian elimination, $\mathbf{A}_{n\times n} = (\prod_{k=1}^{n-1} \mathbf{L}_k)^{-1}\mathbf{U} =: \mathbf{LU}$ $\mathbf{L}_k$: Lower triangular with 1s on the diagonal (by row operation on single column)

1. 如果 **L** 是单位下三角矩阵, 且只有第 k 列有非零元素在对角线下, 则 $\mathbf{L}^{-1}$ 矩阵是 **L** 对角线不动, 其余为相反数.
2. 如果 $\mathbf{L}_1, \mathbf{L}_2$ 矩阵是单位下三角矩阵, 且 $\mathbf{L}_1|\mathbf{L}_2$ 在第 $1 \sim k|k+1 \sim n$ 列有非零元素在对角线下, 则 $\mathbf{L}_1\mathbf{L}_2$ 矩阵是他们的" 拼接".
3. By LU factorization, $\mathbf{A} = \mathbf{LU}$ $\Rightarrow$ By solving $\mathbf{Ly} = \mathbf{b}$ and $\mathbf{Ux} = \mathbf{y}$, we can solve SLE $\mathbf{Ax} = \mathbf{b}$. 用 **FS** 和 **BS** 算法

$$e.g.\ (1)\quad \begin{bmatrix}1&0&0\\2&1&0\\3&0&1\end{bmatrix}^{-1} = \begin{bmatrix}1&0&0\\-2&1&0\\-3&0&1\end{bmatrix} \qquad (2)\quad \begin{bmatrix}1&0&0\\2&1&0\\4&0&1\end{bmatrix}\begin{bmatrix}1&0&0\\0&1&0\\0&5&1\end{bmatrix} = \begin{bmatrix}1&0&0\\2&1&0\\4&5&1\end{bmatrix}$$

Error Analysis of GE: Gaussian elimination is an **unstable** algorithm. (i.e. floating point arithmetic can have a disastrous effect on the computed solution.)

2.3 LUPP | LUCP | GEPP | GECP

Permutation Matrix: For a permutation π, $\mathbf{P}_\pi := [\mathbf{e}_{\pi(1)}, \dots, \mathbf{e}_{\pi(n)}]$ ps: 写的时候是对 I, 第 i 行的 1 移动到第 $\pi(i)$ 行. 注意: 用另一方式定义的 $\mathbf{P}_\pi$ 会让后面对 π 结论不一样

· 左边乘 $\mathbf{P}_\pi$: 交换行 ($\mathbf{P}_\pi$ 的行 代表具体行的交换方法, π 也代表交换行的方法) · 右边乘 $\mathbf{P}_\pi$: 交换列 ($\mathbf{P}_\pi$ 的列 代表具体列的交换方法, π^{-1} 才代表交换列的方法)

· Proposition: $\mathbf{P}_\pi$ is orthogonal, i.e. $\mathbf{P}_\pi^\top = \mathbf{P}_\pi^{-1}$ ($\mathbf{P}_\pi^\top\mathbf{P}_\pi = \mathbf{I}$) · 对于二阶置换矩阵 (Involution): $\mathbf{P}_\pi^2 = \mathbf{I}$, i.e. $\mathbf{P}_\pi^{-1} = \mathbf{P}_\pi$

· **GEPP**: 交换行使每次 $|u_{kk}|$ 最大 **GECP**: 交换行和列使每次 $|u_{kk}|$ 最大 ps: GEPP and GECP are both **stable algorithms** for SLE.

Backward Stable of GEPP: $\forall$ SLE sol $\hat{\mathbf{x}}$ by GEPP, $\exists\Delta\mathbf{A}$ s.t. $(\mathbf{A}+\Delta\mathbf{A})\hat{\mathbf{x}} = \mathbf{b}$ and $\frac{\|\Delta\mathbf{A}\|_\infty}{\|\mathbf{A}\|_\infty} \le p(n)2^{n-1}\varepsilon_m$ $p(n)$ is a cubic polynomial

2.4 QR Factorization Method

QR Factorization: QR factorization of $\mathbf{A}_{n\times n}$ is $\mathbf{A} = \mathbf{QR}$, where **Q** is orthogonal (i.e. $\mathbf{Q}^T = \mathbf{Q}^{-1}$) and **R** is non-singular upper triangular.

· Def of **Q** is orthogonal is equivalent to $\mathbf{Q}^T\mathbf{Q} = \mathbf{I}$. (i.e. columns of **Q** forming an orthonormal set.) · QR produce **large errors**

· **Orthogonal Projection:** $\text{proj}_\mathbf{u}(\mathbf{x}) = \frac{\mathbf{x}\cdot\mathbf{u}}{\|\mathbf{u}\|_2^2}\mathbf{u} = (\mathbf{u}^\top\mathbf{x})\mathbf{u}$ if $\|\mathbf{u}\|_2 = 1$. $\mathbf{x} = \text{proj}_\mathbf{u}(\mathbf{x})^{\text{parallel to } \mathbf{u}} + (\mathbf{x} - \text{proj}_\mathbf{u}(\mathbf{x}))^{\text{orthogonal to } \mathbf{u}}$

Error Analysis of MGS: The matrices $\widehat{\mathbf{Q}}$ and $\widehat{\mathbf{Q}}$ computed by MGS satisfies: $\widehat{\mathbf{Q}}^\top\widehat{\mathbf{Q}} = \mathbf{I} + \Delta\mathbf{I}$ $\widehat{\mathbf{Q}}\widehat{\mathbf{R}} = \mathbf{A} + \Delta\mathbf{A}$ where:

$$\|\Delta\mathbf{I}\|_2 \le \frac{\alpha_1(n)\varepsilon_m\kappa_2(\mathbf{A})}{1-\alpha_1(n)\varepsilon_m\kappa_2(\mathbf{A})}, \text{ if } 1-\alpha_1(n)\varepsilon_m\kappa_2(\mathbf{A}) > 0 \qquad \|\Delta\mathbf{A}\|_2 \le \alpha_2(n)\varepsilon_m\|\mathbf{A}\|_2 \text{ ps: } \alpha_1(n), \alpha_2(n) \text{ 只由 } n \text{ 决定}$$

2.5 Householder QR factorisation

Householder QR: (Most) **stable** algorithm **Householder Reflection Matrix**: $\mathbf{H}(\mathbf{w}) := \mathbf{I} - 2\frac{\mathbf{w}\mathbf{w}^\top}{\|\mathbf{w}\|_2^2}$ $\mathbf{H}(\mathbf{w})\mathbf{x} = \mathbf{x} - 2\text{proj}_\mathbf{w}(\mathbf{x})$

· $\mathbf{H}(\mathbf{w})$ is **orthogonal** and **symmetric** (In the Algorithm is $\mathbf{Q}_k, \mathbf{H}_k$) · $\mathbf{H}(\mathbf{w})\mathbf{x}$ 作用为让 $\mathbf{x}$ 向量对于法向量为 $\mathbf{w}$ 的超平面上进行反射

3 Algorithms

3.1 DP MV MM

DP Dot product $C = \sum_{i=1}^{n} 2 = 2n = \mathcal{O}(n)$

Input: $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$
Output: $s = \mathbf{x} \cdot \mathbf{y} = \mathbf{x}^\top \mathbf{y}$

$s = 0$
for $i = 1, \dots, n$ **do**
  $s = s + x_i y_i$
end for

MV Matrix-vector multiplication
$C = \sum_{i=1}^{m} 2n = 2nm = \mathcal{O}(mn)$

Input: $\mathbf{A} \in \mathbb{R}^{m \times n}, \mathbf{x} \in \mathbb{R}^n$
Output: $\mathbf{y} = \mathbf{A}\mathbf{x}$

for $i = 1, \dots, m$ **do**
  $y_i = \mathbf{a}_i^\top \mathbf{x}$ # DP Algorithm
end for

MM Matrix-matrix multiplication
$C = \sum_{i=1}^{m} \sum_{j=1}^{p} 2n = 2mnp = \mathcal{O}(mnp)$

Input: $\mathbf{A} \in \mathbb{R}^{m \times n}, \mathbf{B} \in \mathbb{R}^{n \times p}$
Output: $\mathbf{C} = \mathbf{A}\mathbf{B}$

for $i = 1, \dots, m$ **do**
  for $j = 1, \dots, p$ **do**
    $c_{ij} = \mathbf{a}_i^\top \mathbf{b}_j$ # DP Algorithm
  end for
end for

3.2 LU FS BS | GE GEPP GECP | LUPP LUCP

LU LU factorisation
$C(n) = \sum_{k=1}^{n-1} (n-k)(1 + 2(n-k+1)) = \frac{2}{3}n^3 + \frac{1}{2}n^2 - \frac{7}{6}n$

Input: $\mathbf{A} \in \mathbb{R}^{n \times n}$
Output: $\mathbf{L}, \mathbf{U} \in \mathbb{R}^{n \times n}$ s.t. $\mathbf{A} = \mathbf{L}\mathbf{U}$

$\mathbf{L} = \mathbf{I}$, $\mathbf{U} = \mathbf{A}$
for $k = 1, \dots, n-1$ **do**
  for $j = k+1, \dots, n$ **do**
    $l_{jk} = u_{jk} / u_{kk}$
    $(u_{jk}, \dots, u_{jn}) = (u_{jk}, \dots, u_{jn}) - l_{jk}(u_{kk}, \dots, u_{kn})$
  end for
end for

FS Forward substitution
$C(n) = \sum_{j=1}^{n} [2(j-1) + 1] = n^2 = \mathcal{O}(n^2)$

Input: $\mathbf{L} \in \mathbb{R}^{n \times n}$ (lower triangular), $\mathbf{b} \in \mathbb{R}^n$
Output: $\mathbf{y} \in \mathbb{R}^n$ such that $\mathbf{L}\mathbf{y} = \mathbf{b}$

for $j = 1, \dots, n$ **do**
  $y_j = \frac{1}{l_{jj}} \left(b_j - \sum_{k=1}^{j-1} l_{jk} y_k \right)$
end for

BS Back substitution
$C(n) = \sum_{j=1}^{n} [2(n-j) + 1] = n^2 = \mathcal{O}(n^2)$

Input: $\mathbf{U} \in \mathbb{R}^{n \times n}$ (upper triangular), $\mathbf{y} \in \mathbb{R}^n$
Output: $\mathbf{x} \in \mathbb{R}^n$ such that $\mathbf{U}\mathbf{x} = \mathbf{y}$

for $j = n, \dots, 1$ **do**
  $x_j = \frac{1}{u_{jj}} \left(y_j - \sum_{k=j+1}^{n} u_{jk} x_k \right)$
end for

GE Gaussian elimination
$C(n) = \frac{2}{3}n^3 + \frac{5}{2}n^2 - \frac{7}{6}n = \mathcal{O}(n^3)$

Input: $\mathbf{A} \in \mathbb{R}^{n \times n}$, $\mathbf{b} \in \mathbb{R}^n$
Output: $\mathbf{x} \in \mathbb{R}^n$ s.t. $\mathbf{A}\mathbf{x} = \mathbf{b}$

Find LU factorisation of $\mathbf{A}$. # LU Algorithm
Solve $\mathbf{L}\mathbf{y} = \mathbf{b}$ using forward substitution. # FS Algorithm
Solve $\mathbf{U}\mathbf{x} = \mathbf{y}$ using back substitution. # BS Algorithm

GEPP Gaussian elimination with partial pivoting
$C(n) = \frac{2}{3}n^3 + 3n^2 - \frac{5}{3}n = \mathcal{O}(n^3)$

Input: $\mathbf{A} \in \mathbb{R}^{n \times n}$, $\mathbf{b} \in \mathbb{R}^n$ # numpy.linalg.solve use it
Output: $\mathbf{x} \in \mathbb{R}^n$ satisfying $\mathbf{A}\mathbf{x} = \mathbf{b}$

Find $\mathbf{P}\mathbf{A} = \mathbf{L}\mathbf{U}$ using Algorithm LUPP # LUPP Algorithm
Solve $\mathbf{L}\mathbf{y} = \mathbf{P}\mathbf{b}$ by forward substitution # FS Algorithm
Solve $\mathbf{U}\mathbf{x} = \mathbf{y}$ by back substitution # BS Algorithm

GECP Gaussian elimination with complete pivoting
$C(n) = n^3 + 3n^2 - 2n = \mathcal{O}(n^3)$

Input: $\mathbf{A} \in \mathbb{R}^{n \times n}$, $\mathbf{b} \in \mathbb{R}^n$
Output: $\mathbf{x} \in \mathbb{R}^n$ satisfying $\mathbf{A}\mathbf{x} = \mathbf{b}$

Find $\mathbf{P}\mathbf{A}\mathbf{Q} = \mathbf{L}\mathbf{U}$ using Algorithm LUCP # LUCP Algorithm
Solve $\mathbf{L}\mathbf{y} = \mathbf{P}\mathbf{b}$ by forward substitution # FS Algorithm
Solve $\mathbf{U}\mathbf{z} = \mathbf{y}$ by back substitution # BS Algorithm
Compute $\mathbf{x} = \mathbf{Q}\mathbf{z}$

LUPP LU factorisation with partial pivoting (not unique)
$C(n) = \frac{2}{3}n^3 + n^2 - \frac{5}{3}n$

Input: $\mathbf{A} \in \mathbb{R}^{n \times n}$
Output: $\mathbf{L}$, $\mathbf{U}$, $\mathbf{P} \in \mathbb{R}^{n \times n}$ s.t. $\mathbf{P}\mathbf{A} = \mathbf{L}\mathbf{U}$ with $\mathbf{L}$ 单位下三角, $\mathbf{U}$ 可逆上三角

$\mathbf{U} = \mathbf{A}$, $\mathbf{L} = \mathbf{I}$, $\mathbf{P} = \mathbf{I}$
for $k = 1, \dots, n-1$ **do**
  Choose $i \in \{k, \dots, n\}$ which maximizes $|u_{ik}|$ # Max Algorithm
  Exchange row $(u_{kk}, \dots, u_{kn})$ with $(u_{ik}, \dots, u_{in})$ # 第 k 行和第 i 行 的 $k \sim n$ 列交换
  Exchange row $(l_{k1}, \dots, l_{k,k-1})$ with $(l_{i1}, \dots, l_{i,k-1})$ # 第 k 行和第 i 行 的 $1 \sim k-1$ 列交换
  Exchange row $(p_{k1}, \dots, p_{kn})$ with $(p_{i1}, \dots, p_{in})$ # 第 k 行和第 i 行 的全部列交换
  for $j = k+1, \dots, n$ **do**
    $l_{jk} = u_{jk} / u_{kk}$
    $(u_{jk}, \dots, u_{jn}) = (u_{jk}, \dots, u_{jn}) - l_{jk}(u_{kk}, \dots, u_{kn})$
  end for
end for

LUCP LU factorisation with complete pivoting (not unique)
$C(n) = n^3 + n^2 - 2n$

Input: $\mathbf{A} \in \mathbb{R}^{n \times n}$
Output: $\mathbf{L}$, $\mathbf{U}$, $\mathbf{P}$, $\mathbf{Q} \in \mathbb{R}^{n \times n}$ s.t. $\mathbf{P}\mathbf{A}\mathbf{Q} = \mathbf{L}\mathbf{U}$ with $\mathbf{L}$ 单位下三角, $\mathbf{U}$ 可逆上三角

$\mathbf{U} = \mathbf{A}$, $\mathbf{L} = \mathbf{I}$, $\mathbf{P} = \mathbf{I}$, $\mathbf{Q} = \mathbf{I}$
for $k = 1, \dots, n-1$ **do**
  Choose $(i, j) \in \{(k, k), \dots, (n, n)\}$ which maximizes $|u_{ij}|$ # Max Algorithm
  Exchange row $(u_{kk}, \dots, u_{kn})$ with $(u_{ik}, \dots, u_{in})$ # 第 k 行和第 i 行 的 $k \sim n$ 列交换 U
  Exchange row $(l_{k1}, \dots, l_{k,k-1})$ with $(l_{i1}, \dots, l_{i,k-1})$ # 第 k 行和第 i 行 的 $1 \sim k-1$ 列交换 L
  Exchange row $(p_{k1}, \dots, p_{kn})$ with $(p_{i1}, \dots, p_{in})$ # 第 k 行和第 i 行 的全部列交换 P
  Exchange column $(u_{1k}, \dots, u_{nk})$ with $(u_{1j}, \dots, u_{nj})$ # 第 k 列和第 j 列 的全部行交换 U
  Exchange column $(q_{1k}, \dots, q_{nk})$ with $(q_{1j}, \dots, q_{nj})$ # 第 k 列和第 j 列 的全部行交换 Q
  for $m = k+1, \dots, n$ **do**
    $l_{mk} = u_{mk} / u_{kk}$
    $(u_{mk}, \dots, u_{mn}) = (u_{mk}, \dots, u_{mn}) - l_{mk}(u_{kk}, \dots, u_{kn})$
  end for
end for

3.3 Max GS MGS SQR | Householder-QR

SQR (SLE) via QR-Factorization $C(n) = 2n^3 + 4n^2 + n$

Input: $\mathbf{A} \in \mathbb{R}^{n \times n}, \mathbf{b} \in \mathbb{R}^n$
Output: $\mathbf{x} \in \mathbb{R}^n$ with $\mathbf{A}\mathbf{x} = \mathbf{b}$

Find QR factorization of $\mathbf{A}$ # GS|MGS Algorithm
Solve $\mathbf{Q}\mathbf{y} = \mathbf{b}$ # MV Algorithm # ps: $\mathbf{y} = \mathbf{Q}^{-1}\mathbf{b} = \mathbf{Q}^\top \mathbf{b}$
Solve $\mathbf{R}\mathbf{x} = \mathbf{y}$ using Algorithm BS # BS Algorithm

(M)GS (Modified) Gram-Schmidt $C(n) = 2n^3 + n^2 + n$

Input: $\mathbf{A} \in \mathbb{R}^{n \times n}$, with columns $\mathbf{a}_1, \dots, \mathbf{a}_n$
Output: $\mathbf{Q} \in \mathbb{R}^{n \times n}$, with orthonormal columns $\mathbf{q}_1, \dots, \mathbf{q}_n$
$\mathbf{R} \in \mathbb{R}^{n \times n}$ non-singular upper triangular matrix; s.t. $\mathbf{A} = \mathbf{Q}\mathbf{R}$

$\mathbf{R} = \mathbf{0}$
for $j = 1, \dots, n$ **do**
  $\mathbf{q}_j = \mathbf{a}_j$
  for $k = 1, \dots, j-1$ **do**
    $r_{kj} = \mathbf{q}_k^\top \mathbf{a}_j \quad | \quad (r_{kj} = \mathbf{q}_k^\top \mathbf{q}_j)$
    $\mathbf{q}_j = \mathbf{q}_j - r_{kj}\mathbf{q}_k$
  end for
  $r_{jj} = \|\mathbf{q}_j\|_2 = \sqrt{\mathbf{q}_j^\top \mathbf{q}_j}$
  $\mathbf{q}_j = \frac{\mathbf{q}_j}{r_{jj}}$
end for

QR Householder QR factorization $C(n) =$

Input: $\mathbf{A} \in \mathbb{R}^{n \times n}$
Output: $\mathbf{Q} \in \mathbb{R}^{n \times n}$ orthogonal
$\mathbf{R} \in \mathbb{R}^{n \times n}$ non-singular upper triangular matrix; s.t. $\mathbf{A} = \mathbf{Q}\mathbf{R}$

$\mathbf{Q} = \mathbf{I}$, $\mathbf{R} = \mathbf{A}$
for $k = 1, \dots, n-1$ **do**
  $\mathbf{u} = (r_{kk}, \dots, r_{nk}) \in \mathbb{R}^{n-k+1}$
  $\mathbf{v} = \mathbf{u} + \text{sign}(u_1)\|\mathbf{u}\|_2 \mathbf{e}_1$
  $\mathbf{w} = \frac{\mathbf{v}}{\|\mathbf{v}\|_2}$
  $\mathbf{H}_k = \mathbf{I} - 2\mathbf{w}\mathbf{w}^\top \in \mathbb{R}^{(n-k+1) \times (n-k+1)}$
  $\mathbf{Q}_k = \mathbf{I}_{k-1} \oplus \mathbf{H}_k$
  $\mathbf{R} = \mathbf{Q}_k \mathbf{R}$
  $\mathbf{Q} = \mathbf{Q}\mathbf{Q}_k$
end for

3.4 Other | Max

Max Finding index of maximum $C(m) = m - 1$

Input: $z \in \mathbb{R}^m$
Output: index i with $|z_i| = \max_{j=1,\dots,m} |z_j|$

$i = 1$, max $= |z_1|$
for $j = 2, \dots, m$ **do**
  if $|z_j| >$ max **then**
    $i = j$
    max $= |z_j|$
  end if
end for

4 Appendix

4.1 Useful Theorems/Formulas

Sum Notation: $\sum_{k=1}^{n} k = \frac{n(n+1)}{2} = \frac{1}{2}n^2 + \frac{1}{2}n$ $\sum_{k=1}^{n} k^2 = \frac{n(n+1)(2n+1)}{6} = \frac{1}{3}n^3 + \frac{1}{2}n^2 + \frac{1}{6}n$ $\sum_{k=1}^{n} k^3 = \left(\frac{n(n+1)}{2}\right)^2 = \frac{1}{4}n^4 + \frac{1}{2}n^3 + \frac{1}{4}n^2$

4.2 Others

Where the error come from calucation:

1. If your calculation involves 15 or more significant digits, you will likely encounter rounding errors. (Since $\varepsilon_m \approx 10^{-16}$ (relative accuracy), which is about 15 significant digits.)
2. **For GE Algorithm**: It is an unstable algorithm. e.g. $\mathbf{A} = [10^{-20}\ 1 \setminus\setminus 1\ 1] \Rightarrow$ By **GE** Algorithm, $\hat{\mathbf{L}}\hat{\mathbf{U}} = \mathbf{A} + [0\ 0 \setminus\setminus 0\ -1]$ (error)
3. **MGS vs. GS**:In floating point arithmetic, the computed vectors $\{\hat{q}_1, \ldots, \hat{q}_{j-1}\}$ will not be orthonormal. Algorithm MGS takes this into account when computing $\hat{q}_j$; Algorithm GS does not.